

Sistemas Operacionais - Trabalho Final

Sistema de troca de mensagens entre múltiplos usuários

Integrantes:

Clara Ernesto de Carvalho - 14559479

Pietra Gullo Salgado Chaves - 14603822

Renan Parpinelli Scarpin - 14712188

Vinicius Neves Gustierrez - 14749363

Objetivo:

O objetivo deste trabalho é projetar e implementar um **sistema de troca de mensagens** que permita a **comunicação em grupo entre múltiplos computadores ou terminais** (inclusive em um mesmo computador). Nesse sistema, cada participante pode enviar mensagens para um servidor central, que as retransmite para todos os demais participantes conectados, assegurando que todas as mensagens enviadas sejam visíveis para o grupo completo. O desenvolvimento deste sistema envolve o uso de conceitos fundamentais de sistemas operacionais, como threads e semáforos, garantindo o gerenciamento de múltiplas conexões simultâneas e a sincronização adequada entre os processos participantes.

Implementação:

O sistema foi **desenvolvido em C++** e é composto por dois programas: **o cliente** e **o servidor**, **cada um utilizando duas threads**. No servidor, há uma região crítica representada por um vetor que armazena os sockets de todas as conexões dos clientes. **Uma das threads do servidor** é responsável por aguardar constantemente novas conexões e, ao recebê-las, adiciona os sockets correspondentes ao vetor. Simultaneamente, **a outra thread** percorre o vetor de conexões para receber as mensagens enviadas pelos clientes e, ao receber uma mensagem, realiza outro percurso no vetor para retransmiti-la a todos os outros clientes conectados. Como a thread responsável por gerenciar as conexões realiza operações de escrita no vetor e a thread de comunicação realiza operações de leitura contínua, foi necessário **utilizar semáforos para evitar que leitura e escrita ocorram simultaneamente**, o que poderia comprometer o funcionamento do sistema. A implementação do semáforo está melhor descrita no vídeo de apresentação, mas foi utilizada a biblioteca **<mutex>** do c++, com as funcionalidades **.lock()** e **.unlock()** que representam as operações de up de down de um semáforo.

No **lado do cliente**, cada programa também é composto por **duas threads**, sendo que uma delas lê as mensagens digitadas pelo usuário no teclado e as envia para o servidor, enquanto a outra recebe mensagens enviadas pelo servidor e as exibe na tela. Como não há uma região crítica sendo acessada simultaneamente por ambas as threads no cliente, **não foi necessário o uso de semáforos nessa parte do sistema**. Essa implementação garante a comunicação entre

o servidor e os clientes, **utilizando threads e semáforos para sincronizar o acesso aos recursos compartilhados no servidor.**

As threads foram implementadas utilizando a biblioteca `<thread>` do c++, com o uso da função `.join()` após a definição de `std::thread`.

Manual de utilização:

Este sistema foi desenvolvido para facilitar a troca de mensagens em grupo entre múltiplos terminais utilizando um servidor central. A seguir, são descritos os passos para compilar e executar o sistema corretamente.

Pré-requisitos

- Certifique-se de ter o compilador `g++`
- O utilitário `make` também deve estar disponível no sistema.

Execução do Sistema

1. Iniciar o Servidor

- Abra um terminal e navegue até a pasta do sistema onde o Makefile está localizado.
- Execute o comando abaixo para compilar e rodar o servidor automaticamente:
 - `make run_server`
- Esse comando compila o código do servidor (se necessário) e iniciará o executável.

2. Iniciar os Clientes

- Em outros terminais, navegue até a mesma pasta do sistema.
- Execute o comando abaixo para compilar e rodar o cliente automaticamente:
 - `make run_client`
- Repita este comando em quantos terminais desejar para abrir múltiplos clientes conectados ao servidor.

3. Enviar e Receber Mensagens

- Cada cliente pode enviar mensagens digitando no terminal e pressionando *Enter*.
- As mensagens enviadas serão retransmitidas pelo servidor para todos os outros clientes conectados, aparecendo automaticamente em seus terminais

Encerrando o Sistema

- Para finalizar o sistema:

- Encerre cada cliente pressionando **Ctrl+C** ou fechando seus respectivos terminais.
- Por fim, encerre o servidor no terminal onde ele está sendo executado pressionando **Ctrl+C**.

Observações Importantes

- O servidor deve ser iniciado antes de qualquer cliente, pois ele gerencia as conexões e a troca de mensagens.
- Não encerre o servidor enquanto houver clientes conectados, pois isso interrompe a comunicação.
- O repositório do GitHub para o trabalho pode ser encontrado em:
<https://github.com/clr-cera/SistemasOperacionais/>

```
renan@DESKTOP-HV30DE1:~/SistemasOperacionais-mai$ ./server
Server Started.
New Connection.
Connection number: 1
New connection.
Connection number: 2
New connection.
Connection number: 3
O terminal superior esquerdo é o servidor
Essa mensagem foi escrita no terminal inferior esquerdo e irá aparecer em todos os terminais
Essa mensagem foi escrita no terminal inferior direito e também irá aparecer em todos os outros terminais

renan@DESKTOP-HV30DE1:~/SistemasOperacionais-mai$ ./client
Connected to Server!
O terminal superior esquerdo é o servidor
Essa mensagem foi escrita no terminal inferior esquerdo e irá aparecer em todos os terminais
Essa mensagem foi escrita no terminal inferior direito e também irá aparecer em todos os outros terminais

renan@DESKTOP-HV30DE1:~/SistemasOperacionais-mai$ ./client
Connected to Server!
O terminal superior esquerdo é o servidor
Essa mensagem foi escrita no terminal inferior esquerdo e irá aparecer em todos os terminais
Essa mensagem foi escrita no terminal inferior direito e também irá aparecer em todos os outros terminais

renan@DESKTOP-HV30DE1:~/SistemasOperacionais-mai$ ./client
Connected to Server!
O terminal superior esquerdo é o servidor
Essa mensagem foi escrita no terminal inferior esquerdo e irá aparecer em todos os terminais
Essa mensagem foi escrita no terminal inferior direito e também irá aparecer em todos os outros terminais
```